EXPERIMENT – 5

PROBLEM 1: IRIS FLOWER CLASSIFICATION

1. Title
Implementation of K-Nearest Neighbours (K-NN) for Iris flower classification.

2. Aim
To classify Iris flowers into Setosa, Versicolor, or Virginica using sepal and petal measurements.

3. Algorithm
1. Start the program.
2. Import the required Python libraries and load/prepare the dataset.
3. Separate the input features (X) and target labels (y).
4. Split the dataset into training and testing sets.
5. Scale the feature values and create a K-NN classifier with K = 3.
6. Train the classifier using the training data.
7. Predict the class labels for the testing data.
8. Calculate and display the classification accuracy and predictions.
9. Stop the program.

4. Python Code
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create and train the K-NN model
```

```
model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=3))
model.fit(X_train, y_train)

# Predict and display results
y_pred = model.predict(X_test)
print("Iris Flower Classification")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("K value: 3")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("First 10 predictions (actual -> predicted):")
for actual, predicted in zip(y_test[:10], y_pred[:10]):
    print(iris.target_names[actual], "->", iris.target_names[predicted])
```

5. Input

Dataset: Iris flower dataset from scikit-learn.

Item	Details
Input features	Sepal length, sepal width, petal length, petal width
Target classes	Setosa (0), Versicolor (1), Virginica (2)
Training samples	120
Testing samples	30
K value	3

6. Output

```
Iris Flower Classification
Training samples: 120
Testing samples: 30
K value: 3
Accuracy: 93.33 %
First 10 predictions (actual -> predicted):
setosa -> setosa
virginica -> virginica
versicolor -> versicolor
versicolor -> versicolor
setosa -> setosa
versicolor -> versicolor
setosa -> setosa
setosa -> setosa
virginica -> virginica
versicolor -> versicolor
```

7. Result

Thus, the K-Nearest Neighbours (K-NN) algorithm was implemented successfully in Python for iris flower classification. The model achieved 93.33% accuracy on the selected test split.

PROBLEM 2: STUDENT PASS/FAIL PREDICTION

1. Title
Implementation of K-Nearest Neighbours (K-NN) to predict student pass/fail results.

2. Aim
To predict whether a student passes or fails based on study hours and attendance percentage.

3. Algorithm
10. Start the program.
11. Import the required Python libraries and load/prepare the dataset.
12. Separate the input features (X) and target labels (y).
13. Split the dataset into training and testing sets.
14. Scale the feature values and create a K-NN classifier with K = 3.
15. Train the classifier using the training data.
16. Predict the class labels for the testing data.
17. Calculate and display the classification accuracy and predictions.
18. Stop the program.

4. Python Code
```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Dataset: [study hours, attendance percentage]
X = np.array([[1,40], [2,45], [2,50], [3,55], [3,60], [4,65],
              [5,70], [5,75], [6,80], [7,90], [1,35], [6,85]],
             dtype=float)
# 0 = Fail, 1 = Pass
y = np.array([0,0,0,0,0,0,1,1,1,1,0,1])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=3))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
print("Student Pass/Fail Prediction")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("K value: 3")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
for actual, predicted in zip(y_test, y_pred):
    print("Actual:", "Pass" if actual else "Fail",
          "| Predicted:", "Pass" if predicted else "Fail")
```

5. Input

Dataset: Small sample student dataset created in the program.

Item	Details
Input features	Study hours and attendance percentage
Target classes	Fail (0), Pass (1)
Training samples	9
Testing samples	3
K value	3

6. Output

```
Student Pass/Fail Prediction
Training samples: 9
Testing samples: 3
K value: 3
Accuracy: 100.0 %
Predictions (actual -> predicted):
Pass -> Pass
Fail -> Fail
Fail -> Fail
```

7. Result

Thus, the K-Nearest Neighbours (K-NN) algorithm was implemented successfully in Python for student pass/fail prediction. The model achieved 100.0% accuracy on the selected test split.

PROBLEM 3: BREAST CANCER CLASSIFICATION

1. Title

Implementation of K-Nearest Neighbours (K-NN) for breast cancer classification.

2. Aim

To classify samples as malignant or benign using the breast cancer dataset.

3. Algorithm

19. Start the program.
20. Import the required Python libraries and load/prepare the dataset.
21. Separate the input features (X) and target labels (y).
22. Split the dataset into training and testing sets.
23. Scale the feature values and create a K-NN classifier with K = 5.
24. Train the classifier using the training data.
25. Predict the class labels for the testing data.
26. Calculate and display the classification accuracy and predictions.
27. Stop the program.

4. Python Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Breast Cancer Classification")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
print("K value: 5")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("First 10 predictions (actual -> predicted):")
for actual, predicted in zip(y_test[:10], y_pred[:10]):
    print(data.target_names[actual], "->", data.target_names[predicted])
```

5. Input

Dataset: Breast cancer Wisconsin dataset from scikit-learn.

Item	Details
Input features	30 numerical cell-nucleus measurements
Target classes	Malignant (0), Benign (1)
Training samples	455
Testing samples	114
K value	5

6. Output

```
Breast Cancer Classification
Training samples: 455
Testing samples: 114
K value: 5
Accuracy: 95.61 %
First 10 predictions (actual -> predicted):
malignant -> malignant
benign -> benign
malignant -> malignant
benign -> malignant
malignant -> malignant
benign -> benign
benign -> benign
malignant -> malignant
malignant -> malignant
malignant -> malignant
```

7. Result

Thus, the K-Nearest Neighbours (K-NN) algorithm was implemented successfully in Python for breast cancer classification. The model achieved 95.61% accuracy on the selected test split.

PROBLEM 4: WINE CLASSIFICATION

1. Title

Implementation of K-Nearest Neighbours (K-NN) for wine class classification.

2. Aim

To classify wine samples into one of three classes using their chemical measurements.

3. Algorithm

28. Start the program.
29. Import the required Python libraries and load/prepare the dataset.
30. Separate the input features (X) and target labels (y).
31. Split the dataset into training and testing sets.
32. Scale the feature values and create a K-NN classifier with K = 5.
33. Train the classifier using the training data.
34. Predict the class labels for the testing data.
35. Calculate and display the classification accuracy and predictions.
36. Stop the program.

4. Python Code

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = load_wine()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Wine Classification")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
print("K value: 5")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("First 10 predictions (actual -> predicted):")
for actual, predicted in zip(y_test[:10], y_pred[:10]):
    print("Class", actual, "-> Class", predicted)
```

5. Input

Dataset: Wine recognition dataset from scikit-learn.

Item	Details
Input features	13 chemical measurements
Target classes	Class 0, Class 1, Class 2
Training samples	142
Testing samples	36
K value	5

6. Output

```
Wine Classification
Training samples: 142
Testing samples: 36
K value: 5
Accuracy: 97.22 %
First 10 predictions (actual -> predicted):
0 -> 0
2 -> 2
0 -> 0
1 -> 1
1 -> 1
0 -> 0
0 -> 0
1 -> 1
1 -> 1
2 -> 2
```

7. Result

Thus, the K-Nearest Neighbours (K-NN) algorithm was implemented successfully in Python for wine classification. The model achieved 97.22% accuracy on the selected test split.

PROBLEM 5: HANDWRITTEN DIGIT CLASSIFICATION

1. Title

Implementation of K-Nearest Neighbours (K-NN) for handwritten digit recognition.

2. Aim

To classify handwritten digit images into digit labels from 0 to 9.

3. Algorithm

37. Start the program.
38. Import the required Python libraries and load/prepare the dataset.
39. Separate the input features (X) and target labels (y).
40. Split the dataset into training and testing sets.
41. Scale the feature values and create a K-NN classifier with K = 3.
42. Train the classifier using the training data.
43. Predict the class labels for the testing data.
44. Calculate and display the classification accuracy and predictions.
45. Stop the program.

4. Python Code

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

digits = load_digits()
X, y = digits.data, digits.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=3))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Handwritten Digit Classification")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
print("K value: 3")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("First 10 predictions (actual -> predicted):")
for actual, predicted in zip(y_test[:10], y_pred[:10]):
    print(actual, "->", predicted)
```

5. Input

Dataset: Digits dataset from scikit-learn (8 × 8 pixel images).

Item	Details
Input features	64 pixel-intensity values
Target classes	Digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Training samples	1437
Testing samples	360
K value	3

6. Output

```
Handwritten Digit Classification
Training samples: 1437
Testing samples: 360
K value: 3
Accuracy: 96.67 %
First 10 predictions (actual -> predicted):
5 -> 5
2 -> 2
8 -> 8
1 -> 1
7 -> 7
2 -> 2
6 -> 6
2 -> 2
6 -> 6
5 -> 5
```

7. Result

Thus, the K-Nearest Neighbours (K-NN) algorithm was implemented successfully in Python for handwritten digit classification. The model achieved 96.67% accuracy on the selected test split.